Contents

Morpheus: On-Device Predictive Autocompletion for Basque Using State Space Models

Preprint — July 2026

Author: Xabier Ezpeleta

Code & Models: `github.com/itzune/morpheus`

Keywords: predictive autocompletion, Basque, Euskara, Mamba, State Space Models, agglutinative languages, on-device inference, keystroke savings, subword tokenization

---

Abstract

Can a Basque text-editor autocompletion system run locally on a consumer device? We answer that **deployment is feasible, but at a quality cost**. We train **Morpheus**, a 91M-parameter Mamba-2 State Space Model on a 4.62B-token curated Basque corpus (~10B tokens seen). The model fits in 55 MB (Q4_K_M), decodes at 318 tok/s on a 2017 laptop CPU (97 ms raw inference; 91.7 tok/s and 87 ms end-to-end through the serving stack, Section 7.2), and achieves 25.3% Character Savings Rate with 76% morpheme boundary accuracy. It is, however, clearly the weakest of the three models we benchmark: a head-to-head comparison against **Kimu 2B (base)** and **Latxa 8B (base)** shows both save +9 CSR points (34.1%/33.2% vs 24.8%), and Morpheus's sweet spot is **sentence-line completion** (email openings, fixed collocations) rather than essay-length generation — a parameter-budget limitation (91M vs 2–8B), not an engineering gap.

This comparison establishes a **two-tier deployment architecture fixed by hardware**: Morpheus is the only model that runs on the edge with acceptable latency (91.7 tok/s on a consumer CPU); both Basque LLMs are GPU-bound, with Kimu 2B matching the 8B quality ceiling at 4x smaller size. Along the way, we expose a **fertility paradox** — lower tokenizer fertility destroys morphological accuracy in agglutinative languages — and a **CSR paradox** — the keystroke-savings metric

structurally penalizes the very language the system is designed to serve. We also contribute five ghost-text inference engineering strategies (smart context, ghost suffix, digit-token repair, garbage filtering, acceptance logging) and a Fill-in-the-Middle extension for cursor-mid-text completion.

---

1. Introduction

Inline ghost-text completion — suggestions accepted with a single keystroke — is the primary interaction paradigm for the two largest deployed LLM products: Gmail Smart Compose and GitHub Copilot. Its dominance reflects measurable productivity: controlled experiments report that AI code completion significantly reduces task time (Peng et al., 2023; Ziegler et al., 2022), and a head-to-head comparison found autocomplete achieves **productivity parity with chat-based assistance at lower interaction friction** — the user never leaves the writing surface (Mozannar et al., 2024). Beyond keystroke savings, ghost-text subtly shapes what people write, nudging toward more predictable language (Arnold et al., 2020). Its defining engineering constraint is real-time latency: Smart Compose targets sub-100 ms per keystroke, beyond which users disengage (Chen et al., 2019). This constraint makes on-device deployment necessary rather than merely desirable — and excludes the languages that cloud-scale systems do not serve.

Production autocompletion systems fall into three paradigms: **server-side multi-token completion** (Gmail Smart Compose, GitHub Copilot), **on-device next-word prediction** (Gboard), and **on-device multi-token continuation** — a Smart Compose equivalent for desktop editors. This third paradigm is the gap: no system runs it on-device for a morphologically complex language. None of the three targets Basque.

Basque (Euskara) is a particularly hard case: a low-resource, morphologically agglutinative language isolate. A single verb can encode subject, object, indirect object, tense, mood, and aspect through suffix chains (e.g., *ekarriko dizkizut* — "I will bring them to you"). Nouns take 12+ case suffixes plus number and definiteness marking. Predicting the next word therefore requires **morphological productivity** — the ability to generate grammatically correct suffix sequences never seen as a unit during training.

Two strategic paths present themselves. **Adapting an existing Basque LLM** (the HiTZ Latxa family, Llama-3.1-based, or Orai NLP's Kimu, Gemma-2-based) is viable for the server tier, but at 2B–8B parameters these models cannot serve the on-device case. **Training a new architecture from scratch** is necessary for the edge tier. We selected Mamba-2 — a State Space Model with O(1) per-step inference and constant memory, the same property Google exploited for Smart Compose but solved at the architecture level rather than with data-center TPUs.

System	Params	Size	Training Data	Deployment	Architecture	Paradigm
Gboard (on-device NWP)	1.4M	1.4 MB	Federated (per-user)	On-device (mobile)	LSTM	Next-word
Gmail Smart Compose	~80M	Server	~8B emails (~320B+ tok est.)	Cloud TPU	LSTM	Multi-token
GitHub Copilot	Multi-B	Server	Proprietary code corpus	Cloud GPU	Transformer (FIM)	Multi-token

System	Params	Size	Training Data	Deployment	Architecture	Paradigm
Morpheus	**91M**	**55 MB**	**4.62B tokens (curated Basque)**	**On-device (laptop)**	**Mamba-2**	**Multi-token**
Kimu 2B (base)	2B	2.1 GB (Q6_K)	ZelaiHandi (1.5B Basque CPT)†	Server (GPU)	Transformer	Multi-token
Latxa 8B (base)	8B	6.6 GB (Q6_K)	Latxa Corpus v2 (4.2B CPT)†	Server (GPU)	Transformer	Multi-token

Table 1. Morpheus in context. Both Google and Morpheus chose recurrent architectures to avoid KV-cache latency; Morpheus achieves it without the data center. The data asymmetry is stark: Smart Compose trains on ~8B proprietary emails (~320B+ tokens, est.) — roughly **70x Morpheus's 4.62B-token curated Basque corpus** — at comparable model scale (91M vs ~80M), yet Morpheus runs the same paradigm on-device.

†Continual pre-training (CPT) atop base models pre-trained on trillions of tokens (Gemma-2-2b ~2T, Llama-3.1-8B ~15T); value shown is Basque-specific CPT only.

---

2. Architecture Selection

Our constraints demand <=300 MB on disk, P90 <=50 ms per-token latency, zero network calls, and training on a single NVIDIA L40 GPU. We evaluated three candidates — xLSTM, a distilled Transformer, and Mamba-2 — and selected **Mamba-2**. It combines LSTM-like inference properties (constant memory, no KV cache) with substantially better language modeling capacity. The distillation path was rejected because Gemma's 256K vocabulary would consume the entire parameter budget in embeddings alone, and KV cache on consumer CPU violates the latency constraint.

---

3. The Tokenizer Decision: Why 4K Vocabulary

The tokenizer is the single most consequential design decision for agglutinative language modeling. Our research synthesized six recent papers (2024–2026) covering 70+ languages, converging on three findings: (1) fertility is misleading for agglutinative tokenizers — low fertility means fused morphemes; (2) Unigram outperforms BPE for agglutinative languages; (3) smaller vocabularies preserve morpheme boundaries.

We trained SentencePiece Unigram tokenizers at 4K, 8K, 16K, and 32K and measured **MorphAcc consistency** — the percentage of test words where the tokenizer places a boundary at the known morpheme split:

Vocab Size	Fertility	MorphAcc
4,000	2.58	**66.7%**
8,000	2.28	61.9%
16,000	2.06	52.4%

Vocab Size	Fertility	MorphAcc
32,000	1.85	28.6%

This replicates the QuechuaTok finding (Contreras, 2026) for a different language family. The degradation is monotonic and accelerating: each vocabulary doubling costs progressively more morphological accuracy. This is the **fertility paradox**: lower fertility (fewer tokens per word) is achieved *exactly by* fusing morphemes into opaque units. The 32K tokenizer memorizes *etxetik* ("from the house") as one token; the 4K tokenizer splits it as `|etxe tik` — root and ablative suffix independently accessible for recombination.

The contrast is most stark in verbal morphology. At 4K, the pluralizer `zki` is an independent reusable token across *dizkizut*, *dakizkioke*, *zitzaizkidan*. At 32K, these are opaque atomic tokens sharing no subword structure. At 91M parameters, the model cannot afford a suboptimal tokenizer — and at 4K, only 3.4% of parameters are embeddings (vs. 27% at 32K), freeing capacity for the SSM layers.

---

4. Training

Parameter	Value
Architecture	Mamba-2 (pure SSM), d_model=768, 24 layers
Total parameters	~91M
Vocabulary	4,000 (SentencePiece Unigram)
Corpus	4.62B tokens, curated from Latxa Corpus v2 (11 of 14 sub-corpora)
Total tokens seen	~10B (~2.16 epochs)
Hardware	1x NVIDIA L40 (48 GB)
Best checkpoint	Step 74K — PPL 7.13 (converged, flat from step 67K)

Data curation was critical. We omitted 3 of 14 sub-corpora — `hplt-v1` (83.8% duplicates), `BOG` (sentence-split legal fragments), and `Aldizkariak` (35% boilerplate) — and applied a four-phase cleaning pipeline (re-parsing, normalization, content filtering, deduplication). An LLM-based audit rated retained sources 4.6/5. Validation leakage prevention excluded 68,755 held-out lines from training.

Convergence analysis revealed that the model effectively stopped learning at ~8.8B tokens — the final ~1.2B tokens produced no measurable PPL improvement. The improvement rate dropped 8.8x between the first and second halves of training. With ~20–30% corpus noise, the effective high-quality data is ~7–8B tokens, matching the convergence point. **For low-resource languages, data quality is the binding constraint, not quantity** — a 2–5B token high-quality corpus would likely match the current 10B mixed-quality one.

---

5. Inference Engineering for Ghost-Text Autocomplete

Deploying a subword language model as real-time ghost text in a desktop editor (the Obsidian plugin and web demo) exposes failure modes invisible in batch perplexity evaluation. The server-side proxy applies five strategies:

1. **Smart context (token-aligned prefix).** When the user types *Kaixo, zer mod*, the SentencePiece tokenizer segments the trailing fragment as `[|mo, d]` — a partial token on a different path than the complete word *moduz*. The proxy strips trailing subword fragments before querying, giving the model a clean token boundary.
2. **Ghost suffix (overlap deduplication).** Mirroring Gmail Smart Compose, only the non-typed suffix is shown as ghost text. If the user typed *mod* and the model predicts * moduz?*, the ghost text is* uz?*.
3. **Digit-token repair.** SentencePiece byte-fallback can produce digit tokens that decode to garbage. Rather than filtering post-hoc, the proxy walks the greedy token path and swaps digit tokens for their best non-digit alternative at each position — a token-level repair before decoding.
4. **Byte-fallback garbage filtering.** When the typed prefix's tokenization is incompatible with the desired word (e.g. *zaud* -> `[|za, u, d]` cannot reach *zaude*), the model emits non-Latin byte-fallback characters. The proxy detects and suppresses these, falling back to multi-prefix querying when needed.
5. **Confidence scoring and acceptance logging.** Each suggestion carries an average per-token probability (exposed to the Obsidian plugin's `confidenceThreshold` setting). Every Tab-acceptance is logged to `/api/log`, transforming real sessions into an evaluation dataset.

Evaluation methodology. The product metric is Character Savings Rate (CSR) under free-acceptance (Trnka & McCoy, 2008) — the same methodology Gmail Smart Compose uses (Chen et al., 2019): a keystroke-by-keystroke simulation where Tab-accepts save matched characters. Our CSR (Section 7) is measured on the deployed greedy endpoint with all five strategies active. A companion next-word keyboard paradigm (word chips, retokenization fallback, sticky merge) is documented in the futo-basque project.

---

6. Deployment: From Model to Editor

A model that achieves 318 tok/s decode in isolation cannot reach the user's cursor without a serving stack — and that stack imposes real cost: end-to-end throughput drops to 91.7 tok/s (87 ms per request, Section 7.2), with the serving stack (Python FastAPI, SentencePiece tokenization, inference-engineering pipeline) adding modest overhead over raw inference. Morpheus deploys through a **thick-proxy architecture**: a FastAPI server wraps a compiled `llama.cpp` backend and exposes a thin-client protocol that any editor can speak.

Export and quantization. The PyTorch checkpoint is exported to GGUF via `llama.cpp` tooling and quantized to Q4_K_M (55 MB). A critical reproducibility finding: `llama-server` auto-prepends a BOS token for string prompts, and its built-in SentencePiece tokenizer diverges from the reference library on the 4K vocabulary, collapsing CSR from ~28% to ~4%. The server therefore sends **token IDs, not strings** — a mitigation that generalizes to FIM.

The thick proxy. A FastAPI server (`demo/server.py`) wraps a compiled `llama.cpp` binary and holds the SentencePiece tokenizer, the FIM template, and the inference-engineering strategies from Section 5. It exposes a convenience route for thin clients:

```
POST /v1/complete  {prefix, suffix}  ->  {text, confidence}
```

Any editor plugin speaks this protocol — no FIM tokens, no tokenizer, no SentencePiece. The server applies the FIM template, encodes to token IDs, calls `llama-server`, and post-processes (garbage filtering, confidence scoring, candidate extraction). A Docker deployment (`demo/Dockerfile`) compiles `llama.cpp` with Mamba-2 SSM support and runs on CPU: on a 2017 laptop, the 55 MB model achieves ~160 ms per FIM request.

Desktop integration. We built an Obsidian plugin (`demo/obsidian-morpheus-plugin/`) that renders inline ghost text via CodeMirror 6. On typing pause, it POSTs the text before and after the cursor to `/v1/complete`; the response appears as transparent inline text (**Tab** accepts, **Esc** dismisses). The plugin is **backend-agnostic**: only the Server URL setting differs between tiers. At `localhost:9090` it uses the on-device Mamba-2 model; at a GPU server URL it uses Latxa 8B — no plugin modification, no reinstall.

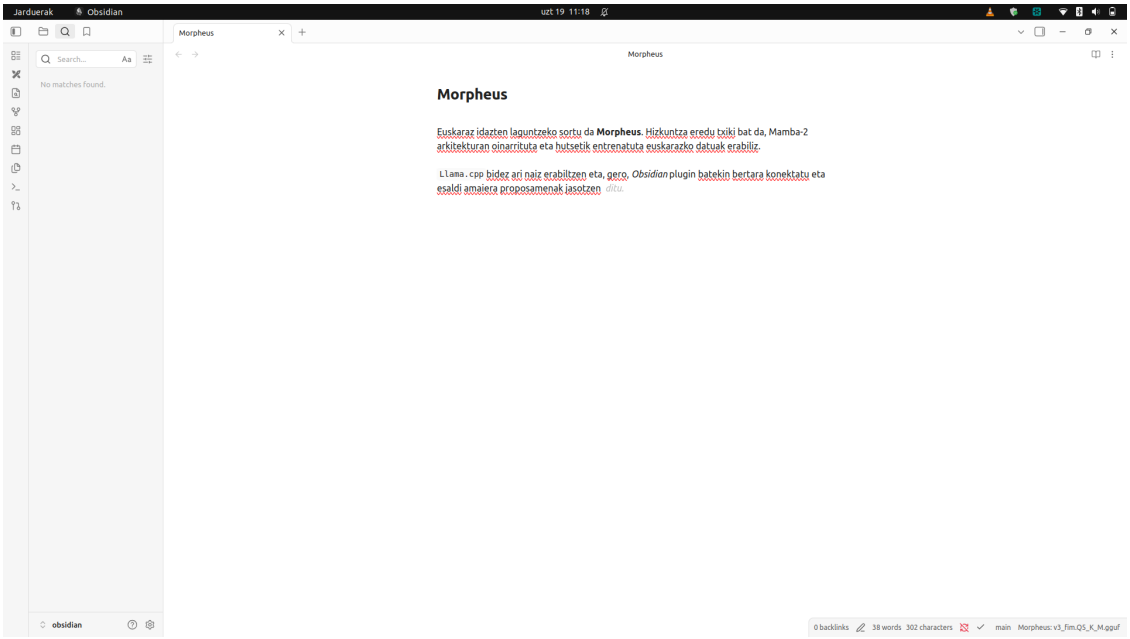

Figure 1: **Figure 1.** Morpheus on-device: ghost-text autocomplete in Obsidian, served by the local 91M Mamba-2 model (55 MB, CPU). The status bar confirms the local model. The user is writing about the deployment itself ("...Obsidian plugin batekin... proposamenak jasotzen" — "with an Obsidian plugin... receiving suggestions") and the model completes the verb *ditu*.

The figures make the deployment architecture visible: identical client, different backend, abstracted behind a URL. The 91M model runs locally (status bar: `v3_fim.Q5_K_M.gguf`); the 2B and 8B models run on the GPU (status bar: `Gemma-Kimu-2b-base.Q6_K.gguf` or `HITZ.Latxa-Llama-3.1-8B.Q6_K.gguf`). The hardware constraint — not a preference — determines which tier serves the user.

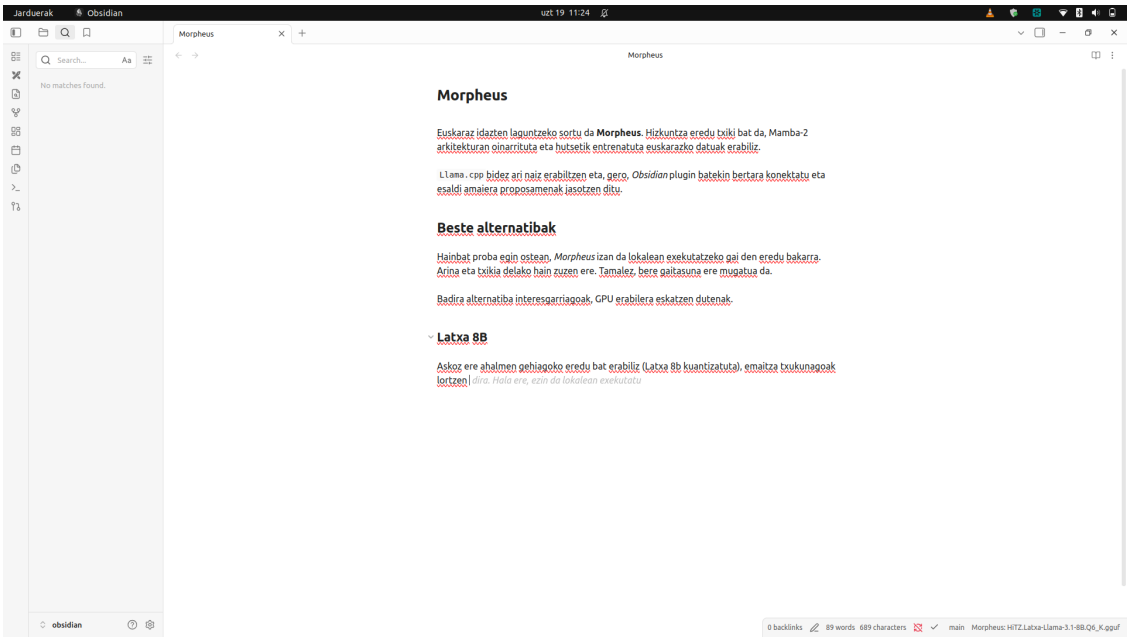

Figure 2: **Figure 2.** The same plugin, same editor — pointed at a GPU server running Latxa 8B (6.6 GB). Only the Server URL changed. The suggestion is longer and more specific. The model completes the user's sentence about Latxa with "*dira. Hala ere, ezin da lokalean exekutatu*" — "they are. However, it cannot be run locally" — a self-aware articulation of the two-tier constraint.

7. Evaluation and Results

7.1 Core Metrics

Metric	Result	Interpretation
Held-out PPL	7.13	Strong LM quality; converged
CSR (macro, n=300)	25.26% [24.0%, 26.5%]	Meaningful keystroke savings
MorphAcc	76%	3.8x improvement over 32K tokenizer
BPC	0.970	Matches GPT-2 eus-euscrawl (0.981) at 27% fewer params
Q4_K_M size	55 MB	Well within 300 MB budget

PPL is the only reliable checkpoint-ranking metric. It improved monotonically across all 14 evaluation files (7.56 -> 7.17 -> 7.13). CSR, MorphAcc, and case paradigm metrics all saturated or produced noisy, non-monotonic signal. CSR is fragile to implement (string prompts gave ~4% vs ~28% with token-ID prompts due to BOS/tokenizer bugs) and cannot distinguish checkpoints at n=300 (all confidence intervals overlap).

7.2 Cross-Model Comparison

Bits Per Character (BPC) is the correct tokenizer-independent metric for comparing models with different vocabularies. BPC, simplified CSR, and Word Accuracy below are computed via direct PyTorch forward passes on full-precision (BF16) HuggingFace weights (`scripts/eval_baselines.py`) — not the quantized GGUF deployment:

Model	Params	BPC	CSR (simplified)	Word Accuracy
GPT-2 eus-euscrawl	124M	0.981	0.110	37.6%
Morpheus (Mamba-2)	91M	0.970	0.094	60.4%
Kimu 2B (base)	2B	0.744	0.215	61.7%
Latxa 8B (base)	8B	**0.490**	**0.266**	**75.2%**

Morpheus matches GPT-2's BPC at fewer parameters (the difference is primarily attributable to 11x more training data). **The Basque LLM comparison** fixes the deployment architecture: both Kimu 2B (Orai NLP, Gemma-2 CPT) and Latxa 8B (HiTZ, Llama-3.1 CPT) save +9 free-acceptance CSR points over Morpheus (34.1%/33.2% vs 24.8%, measured on Q6_K quantized GGUF served via the full demo stack) with artifact-free BPE output, but are GPU-bound. On raw simplified CSR (table above), Latxa 8B leads as expected for a 4x larger model; on the free-acceptance benchmark, Kimu 2B *edges out* Latxa 8B at 4x smaller size — a 2B Basque-pretrained model reaches the 8B quality ceiling on this task. **A critical capability gap**: Morpheus has been extended with FIM (Section 8) and serves cursor-mid-text infill; the base Basque LLMs have **no FIM training** — they emit EOS immediately on FIM sentinels, so their +9 CSR advantage holds only for end-of-buffer append. Closing the infill gap via FIM continued pretraining on Kimu 2B is the natural next step (Section 9). On the consumer laptop CPU, neither Basque LLM is viable: Kimu collapses to 5.6 tok/s (1,439 ms/request, 9.6x over budget), Latxa to 2.8 tok/s (2,869 ms/request, 19x over), while Morpheus sustains 91.7 tok/s.

Hardware	Model	Latency	tok/s	Memory
L40 (GPU)	**Morpheus Q5_K_M**	**70 ms**	**114.1**	**602 MiB VRAM**
L40 (GPU)	Kimu 2B Q6_K	74 ms	107.6	3,036 MiB VRAM
L40 (GPU)	Latxa 8B Q6_K	102 ms	78.3	6,992 MiB VRAM
i7-8550U (CPU)	**Morpheus Q5_K_M**	**87 ms**	**91.7**	**264 MiB RAM**
i7-8550U (CPU)	Kimu 2B Q6_K	1,439 ms	5.6	2,357 MiB RAM
i7-8550U (CPU)	Latxa 8B Q6_K	2,869 ms	2.8	6,648 MiB RAM

The qualitative difference is clear: both Kimu and Latxa commit to semantically specific continuations (a concrete meeting time, an encryption property), while Morpheus often drifts into high-frequency connective filler or unrelated statistical patterns. Morpheus's sweet spot is **formulaic completion** — email openings, fixed collocations, administrative phrasing — and **domain-specialized fine-tunes**.

7.3 The CSR Paradox

A typing simulation with 15 parallel sentences (5 per language, identical semantic content) revealed that the model's native Basque achieves the **lowest** simulated CSR:

Language	Top-3 Accuracy	Simulated CSR	Avg. prefix before acceptance
Basque (native)	**79.5%**	**19.6%**	**4.4 chars**
English (<1% corpus)	72.0%	25.5%	2.7 chars
Spanish (<1% corpus)	73.9%	29.3%	3.1 chars

This is not a model deficiency — it is a structural property of agglutinative morphology. Basque words are longer, so the user must type more characters before the model can confidently predict the correct form. A word like *paseatzera* ("to go for a walk") cannot be predicted from *pa*; the model needs *paseatzer* before converging. Meanwhile, English *walk* is predictable from *w* given sufficient context. Basque achieves the **highest** Top-3 accuracy (79.5%) — the model identifies the correct word more often — but the keystroke savings are consumed by the longer prefix.

CSR penalizes the very language such systems are designed to serve and should not be used as a primary optimization target. This aligns with GitHub's finding that acceptance-rate optimization "could lead to incorrectly favoring a high volume of simple and short suggestions" (Fu & Mogensen, 2025).

7.4 The Metric Inversion: Exact-Match Metrics Move Opposite to PPL

The CSR paradox (Section 7.3) shows that CSR penalizes agglutinative languages *structurally* — a cross-language effect. A more troubling question is whether autocomplete metrics track model quality *within* a single language, *across training*. To test this, we ran the next-word keyboard simulation (PyTorch port of the deployed algorithm; futo-basque companion) on **seven checkpoints** spanning the full training trajectory (step 10K–76K), using the same 30 Basque test sentences at every checkpoint. The three checkpoints with held-out PPL measurements tell the story:

Step	Held-out PPL	NW-CSR	Top-1 Acc	Top-3 Acc	Confidence
32K	7.56	0.396	0.523	0.866	0.401
54K	7.17	0.385	0.503	0.859	0.416
74K	**7.13**	**0.361**	**0.503**	**0.832**	**0.433**

The model improved (PPL 7.56 -> 7.13), yet *every* exact-match metric moved the wrong way: NW-CSR, Top-1, and Top-3 all **decrease**. The only metric that improves monotonically is **confidence** (0.401 -> 0.433) — the model becomes more certain about its predictions, but not more accurate at matching the gold-standard word.

Cross-backend validation. We recomputed the metrics from stored GGUF simulation events (the demo server logs full ranked candidate lists). The inversion is **partially backend-dependent**: NW-CSR inverts only in PyTorch (0.396 -> 0.361, ↓) while GGUF tracks PPL (0.362 -> 0.389,

↑) — likely a bf16 forward-pass artifact. But **Top-1 accuracy decreases in both backends** (PyTorch: 0.523 -> 0.503; GGUF: 0.537 -> 0.510) — a real effect, not a computation artifact.

Hypothesis. As the model improves, it distributes probability across multiple valid morphological continuations (*paseatzera*, *paseatzeko*, *paseatzen*), depressing top-1 on any single one. PPL captures this distributional sharpening; exact-match Top-K cannot. In GGUF, the gold word still appears in the top-3 (Top-3 flat), so the user can still accept it after a few more characters — keeping NW-CSR roughly tracking PPL. In PyTorch (bf16), Top-3 also decreases — the gold word drops out of the candidate list entirely, a more severe degradation partly attributable to precision.

Practical implication. No exact-match autocomplete metric should be used as a primary checkpoint selection criterion for agglutinative autocomplete. PPL is the only metric that reliably tracks model quality. All CIs overlap (n=30); the trends are directional, not statistically proven. Full cross-backend data in Appendix B of the detailed version.

8. Fill-in-the-Middle (FIM) Extension

The AR-only model can only extend a prefix. For desktop text editing, the cursor often sits within a sentence, requiring text that bridges what precedes and follows — the **Fill-in-the-Middle** objective. We extended Morpheus via continued pre-training from the step-74K checkpoint, using Code Llama-style FIM tokens (`<PRE>`, `<SUF>`, `<MID>`, `<EOT>`), token-level splitting, and a 500M-token budget.

The key engineering finding is the stop-token reliability problem. A naive 50/50 FIM/AR mix produces coherent infill but fails to reliably emit `<EOT>` (~77% emission), causing over-generation and deeply negative keystrokes saved (~-25%). The root cause is signal sparsity — `<EOT>` is a single token per FIM example. A **5x loss weight on `<EOT>`** with a **70/30 FIM/AR ratio** resolves this:

Metric	Before	After (5x EOT weight)
`<EOT>` emission	~77%	88.4%
Keystrokes saved	-25%	-5.9%
AR valid PPL	7.13	7.5 (stable)
Premature truncation	—	1.4% (< 15% threshold)

The feared premature-truncation failure mode did not materialize. AR capability is preserved ("FIM-for-free"), and the model now slightly under-generates (40.3 vs. 45.0 chars) — a preferable failure mode for users.

---

9. Known Limitations

- **CSR of 25%** is below the ~80% achievable in English autocomplete — partly model quality, partly the structural CSR paradox.
- **Corpus-induced artifacts:** The model over-predicts dates and numbers because the corpus is dominated by encyclopedic and journalistic prose. *Aipatu bezala, -> 2015eko ekainean,* instead of a general continuation. This is domain mismatch: Smart Compose avoids it by training on emails and deploying for emails; for Basque, no large conversational corpus exists.

- **No morphological pre-segmentation** yet: MorphAcc could improve from 76% toward 83%+ with Apertium-based surface-preserving boundaries.
- **No user study:** All evaluation is simulation-based. The model is too large for mobile (Gboard: 1.4M/1.4 MB); a distilled ~5–10M variant has not been trained.
- **Evaluation limitations:** PPL is the only reliable metric; CSR and MorphAcc saturate at available sample sizes. The metric inversion (Section 7.4) — autocomplete metrics moving opposite to PPL improvement — suggests that a better model distributes probability across valid morphological variants, depressing exact-match accuracy.
- **Serving overhead:** Raw decode speed (318 tok/s, Section 5.2) is ~3.5x faster than end-to-end served throughput (91.7 tok/s, Section 7.2). Part of this gap is measurement methodology (decode-only vs. total-wall including prefill) and quantization (Q4_K_M raw vs. Q5_K_M served); the remaining overhead is Python FastAPI + SentencePiece tokenization + inference-engineering pipeline. Native C++ tokenization and streaming responses could further reduce this gap.

---

10. Conclusion

Morpheus demonstrates that **on-device predictive autocompletion for an agglutinative language is feasible**. A 91M Mamba-2 model, fitting in 55 MB with zero network calls, provides real-time ghost-text completion on a 2017 laptop CPU — comparable in scale to Gmail Smart Compose but without the data-center dependency.

An honest quality assessment. Morpheus is clearly the weakest of the three models we benchmarked (Section 7.2). It trails Kimu 2B and Latxa 8B by ~9 CSR points (24.8% vs 33–34%) and by 0.23–0.48 BPC. The qualitative gap is more telling than the numbers: on essay and technical prompts, Morpheus drifts into high-frequency connective filler or unrelated statistical patterns rather than holding a sentence's semantic thread, while both Basque LLMs commit to concrete, on-topic continuations. Morpheus's sweet spot is **sentence-line completion** — email openings, fixed collocations, administrative phrasing — where local statistics suffice. It lacks the context comprehension to sustain coherent multi-sentence argumentation; no amount of inference engineering closes that gap, because it is a parameter-budget limitation (91M vs 2–8B). What Morpheus buys with that constraint is the only model that runs on-device with acceptable latency — a hardware constraint, not a quality claim.

Key Contributions

1. **The fertility paradox.** For agglutinative tokenizers, lower fertility *destroys* morphological accuracy. MorphAcc drops from 66.7% (4K) to 28.6% (32K) — replicating QuechuaTok across language families.

2. **The CSR paradox.** Keystroke-savings metrics structurally penalize morphologically complex languages. The model's native Basque achieves the *lowest* CSR despite the *highest* Top-3 accuracy. CSR should never be used as a primary optimization target for agglutinative autocomplete.

3. **Inference engineering for ghost-text autocomplete.** Five server-side strategies — smart context (token-aligned prefix), ghost suffix (Smart Compose overlap dedup), digit-token repair, byte-fallback garbage filtering, and confidence scoring with acceptance logging — that address

failure modes of subword-tokenized SSM models in real-time ghost-text deployment. The free-acceptance CSR (Trnka & McCoy, 2008; Chen et al., 2019) is the primary product metric, measured on the deployed endpoint with all strategies active.

4. **Two-tier deployment architecture.** Morpheus (91M, 55 MB) runs on the edge for formulaic completion and domain fine-tunes. Kimu 2B (2.1 GB) and Latxa 8B (6.2 GB) are the server-side quality ceiling (+9 CSR points, cross-domain competence), but GPU-bound. Kimu 2B is the efficiency frontier: it matches Latxa's CSR at 4x smaller size. The split is a hardware constraint, not a preference. A thick-proxy FastAPI server wraps compiled `llama.cpp` and exposes a `{prefix, suffix} -> {text}` protocol; an Obsidian plugin demonstrates that the identical editor client switches tiers by changing one URL (Section 6).
5. **Data quality over quantity.** The model converged at ~8.8B tokens; a 2–5B token high-quality corpus would likely match the full 10B mixed-quality one. For low-resource languages, aggressive quality filtering dominates raw scale.
6. **FIM stop-token engineering.** The `<EOT>` signal is too sparse for reliable learning within modest CPT budgets. A 5x loss weight resolves over-generation without inducing premature truncation.

The Path Forward

The model has converged. The next steps are: (1) integrate Apertium Basque for morpheme pre-segmentation; (2) distill a mobile-variant model (~5–10M); (3) run FIM continued pretraining on Kimu 2B (base) as the server-side model — the leading candidate, as it matches Latxa 8B's CSR at 4x smaller size — with Latxa 8B as fallback; (4) domain-specific fine-tuning to mitigate corpus-induced artifacts; and (5) conduct user studies with real Basque speakers.

Appendix E: Three-Model Qualitative Comparison

Top-3 sampled completions (temperature 0.7, 20 tokens) from each model. All prompts are **authentic Basque sentences** sourced from Wikipedia (eu), Berria newspaper, and the iberba.eus email-writing guide — no invented text. Gold continuations are the actual completions from the source documents. No evaluation is offered here; the reader is invited to compare.

Models: **Morpheus** (91M Mamba-2, Q5_K_M), **Kimu 2B** (Gemma-2, Q6_K), **Latxa 8B** (Llama-3.1, Q6_K). All served via the same demo stack on an L40 GPU.

Three representative examples shown here. The full set of 7 AR + 5 FIM examples is in Appendix E of the detailed version.

E.4 Geography — Wikipedia (Bilbo)

> **Prompt:** Euriak udazkenean eta udaberrian ugariak izaten dira; negua, berriz, (Rains are abundant in autumn and spring; winter, however,) **Gold:** eztia, eta uda ez oso beroa. (mild, and the summer not very hot.)

#	Morpheus	Kimu 2B	Latxa 8B
1	elurtutako lurraldeetan egoten da. (0.39)	lehorra izaten da. Euri urtea izaten denean, uholdeak izaten (0.517)	hotz eta hezea da. Urteko batez besteko tenperatura (0.611)
2	freskoa, eta udazkenean, berriz, ezberdina, baina (0.363)	lehorra izaten da. Horrez gain, mendiek eta basoek udazk (0.525)	hotza eta elurtsua izaten da. Udak epelak eta hezeak (0.634)
3	udazkenean eta udaberrian izaten da, eta negu-neguetan, uda (0.4)	lehorragoa izaten da. Abeltzaintzari dagokionez, ab (0.595)	hotz eta lehorrak izaten dira. Udak epelak izaten dira (0.59)

E.7 Email / workplace — iberba.eus (Epostak eta gutunak)

Prompt: Iazko azken batzarrean adostu genuen bezala, urteko memoria (As agreed in last year's meeting, the annual report) **Gold:** martxoa baino lehen amaitu beharra dago. (must be finished before March.)

#	Morpheus	Kimu 2B	Latxa 8B
1	aurkeztuko dugu. (0.377)	prestatzen hasiak gara. Helburu nagusia herritar guztiek gure lanaren (0.365)	eta balantzea eginda ekarri ditugu, egia esan, ez da (0.367)
2	egitea, eta beti ere, talde bakoitzak bere testuinguruan Plan Orokorraren barruan, nola ez, bere (0.191)	idazteko ardura hartu nuen nire gain, eta gaur, urte (0.488)	idatzi dugu. Memorian urtean zehar egindako ekintza (0.521)
3	ere osatu eta aurkeztu dugu. (0.182)	eta balantzea aurkeztu dizkizuegu. (0.567)	egitea tokatzen zaigu, eta proiektuaren hasieran jarritako (0.374)

FIM (Fill-in-the-Middle) Examples

The prefix and suffix are given; the model must generate the **middle** (shown in bold in the full sentence). Only **Morpheus** was trained with FIM. Kimu 2B and Latxa 8B are base models without FIM training — they cannot attend to the suffix and typically generate from the prefix only, emit empty strings, or leak FIM sentinel tokens. Their columns are included to illustrate this capability gap (cf. Section 6.6 / Section 7.2).

F.5 Email / workplace — iberba.eus (Epostak eta gutunak)

Full sentence: Iazko azken batzarrean adostu genuen bezala, **urteko memoria martxoa baino lehen** amaitu beharra dago.

#	Morpheus (FIM)	Kimu 2B (no FIM)	Latxa 8B (no FIM)
1	`aurten ere,` (0.167)	`Jarraitu irakurtzen 'Ostiela!' blogean.` (0.545)	*(empty)*
2	`aurten ez da ezer` (0.31)	`Hala, bihar, martxoak 25, asteazkena, eging` (0.433)	`Ezin daiteke onartu, ezin daiteke onetsi.<|f` (0.379)
3	`hileko kuotak` (0.272)	`Kultura batzordeak egindako proposamena da.` (0.425)	*(empty)*

References

1. Contreras, M. (2026). *QuechuaTok: Morphological Boundary Accuracy as a Necessary Metric for Tokenizer Evaluation in Agglutinative Low-Resource Languages*. arXiv:2606.23943.
2. Chen, M. X., et al. (2019). *Gmail Smart Compose: Real-Time Assisted Writing*. arXiv:1906.00080.
3. Fu, S., & Mogensen, J. (2025). *The Road to Better Completions: Building a Faster, Smarter GitHub Copilot*. GitHub Blog.
4. Gu, A., & Dao, T. (2023). *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. arXiv:2312.00752.
5. Dao, T., & Gu, A. (2024). *Transformers are SSMs: Generalized Models and Efficient Algorithms through Structured State Space Duality*. arXiv:2405.21060.
6. Etxaniz, J., et al. (2024). *Latxa: An Open Language Model and Evaluation Suite for Basque*. arXiv:2403.20266.
7. Bavarian, M., et al. (2022). *Efficient Training of Language Models to Fill in the Middle*. arXiv:2207.14255.
8. Trnka, K., & McCoy, K. (2008). *Evaluating Word Prediction: Framing Keystroke Savings*. ACL 2008.
9. Lane, W., Harrigan, A., & Arppe, A. (2022). *Interactive Word Completion for Plains Cree*. ACL 2022.
10. Xu, N., & Kim, A. (2026). *Tokenization and Morphological Fidelity in Uralic NLP*. arXiv.
11. Stephen, A., & Libovický, J. (2026). *Evaluating Morphological Plausibility of Subword Tokenization*. arXiv.
12. Hoffmann, J., et al. (2022). *Training Compute-Optimal Large Language Models* (Chinchilla). arXiv:2203.15556.
13. Sardana, N., et al. (2024). *Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws*. arXiv:2401.00448.
14. Roziere, B., et al. (2023). *Code Llama: Open Foundation Models for Code*. arXiv:2308.12950.
15. Mozannar, C., et al. (2024). *The RealHumanEval: Evaluating Large Language Models' Abilities to Support Programmers*. arXiv:2404.02806.

16. Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). *The Impact of AI on Developer Productivity: Evidence from GitHub Copilot.* arXiv:2302.06590.
17. Ziegler, A., et al. (2022). *Productivity Assessment of Neural Code Completion.* MAPS@PLDI 2022.
18. Arnold, K., et al. (2020). *Predictive Text Encourages Predictable Writing.* ACM IUI 2020.

Current as of July 2026 — AR training complete (76,294 steps, best checkpoint step 74,000, PPL 7.13); FIM continued pre-training complete.